

CQt day3 hw1 레포트(알고리즘 보고서)

로봇 예비단원 주호진

1. 과제

알고리즘 보고서

작성 폰트: 맑은고딕 / 제목: 13pt / 본문: 10pt /

부제목, 목차 등: 10~12pt 목차, 표지, 인사말 제외 최소 10p 이상

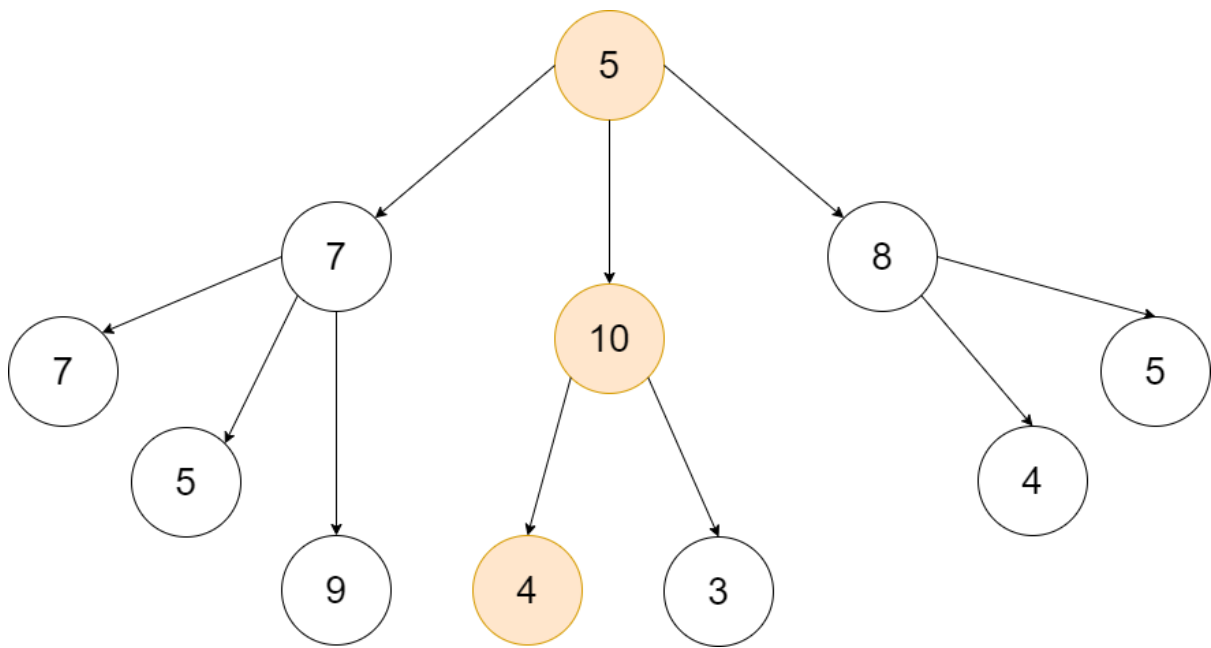
개조식으로 작성

2. 알고리즘

2-1. 다익스트라 알고리즘(Dijkstra Algorithm)

- 음의 가중치가 없는 그래프의 한 노드에서 각 모든 노드까지의 최단 거리를 구하는 알고리즘
- 그리디 알고리즘과 다이나믹 프로그래밍 기법을 사용한 알고리즘이라고 볼 수 있음

2-1-1. 그리디 알고리즘

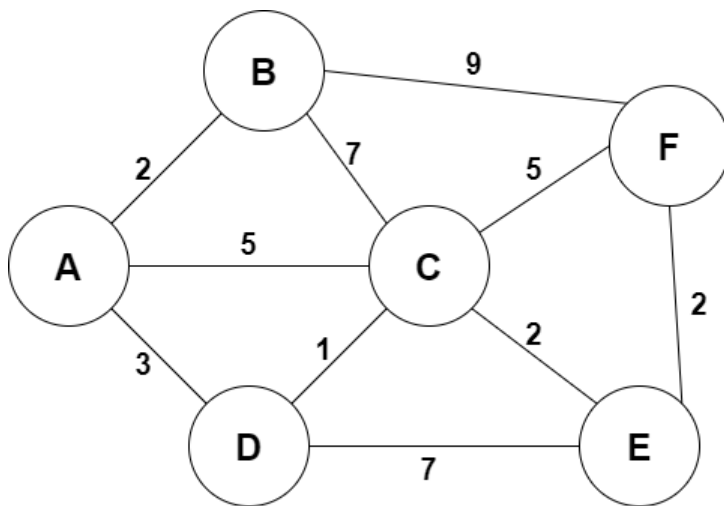


- 매 선택에서 지금 당장 최적인 답을 선택하는 알고리즘
- 선택의 순간순간에 대해서는 최적이지만, 종합적으로 봤을 때 최적인 것은 아님
- 위 사진으로 봤을 때 노드 값을 최대로 구한다면 5, 7, 9로 총합 21이지만 그리디 알고리즘에서는 5에서의 선택지가 7, 8, 10이기 때문에 5인 순간에서의 최적인 10을 선택하게 됨
- 최종 결과값으로 보면 $5+10+4$ 이므로 19의 값을 갖게 됨

2-1-2. 다이나믹 프로그래밍(DP)

- 큰 문제를 작은 문제로 나눈 뒤, 기억하여 푸는 기법
- 공간 복잡도(메모리 공간)를 늘려 시간 복잡도(연산 횟수)를 줄이는 방식
- Top Down 방식은 작은 문제를 해결하면서 해결한 문제의 결과를 저장하여 같은 문제가 나왔을 때 재활용 하는 것
- Bottom Up 방식은 작은 문제부터 차례대로 풀어 그 값을 통째로 저장하는 방식

A에서 F까지의 최단 경로를 구하는 다익스트라 알고리즘의 예시

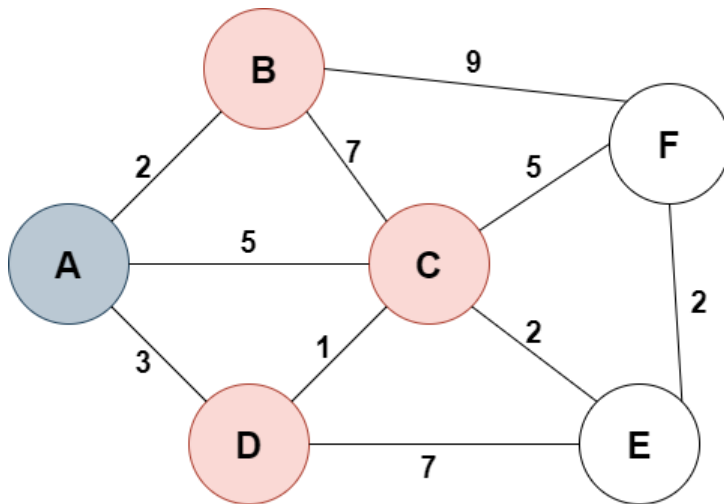


$S = \{ \}$

$d[A] = 0$
 $d[B] = \text{INF}$
 $d[C] = \text{INF}$
 $d[D] = \text{INF}$
 $d[E] = \text{INF}$
 $d[F] = \text{INF}$

$Q = \{A, B, C, D, E, F\}$

S는 방문한 노드의 집합, $d[N]$ 은 $A \rightarrow N$ 의 최단 거리값, Q는 방문하지 않은 노드의 집합



$S = \{A\}$

$d[A] = 0$
 $d[B] = 2$
 $d[C] = 5$
 $d[D] = 3$
 $d[E] = \text{INF}$
 $d[F] = \text{INF}$

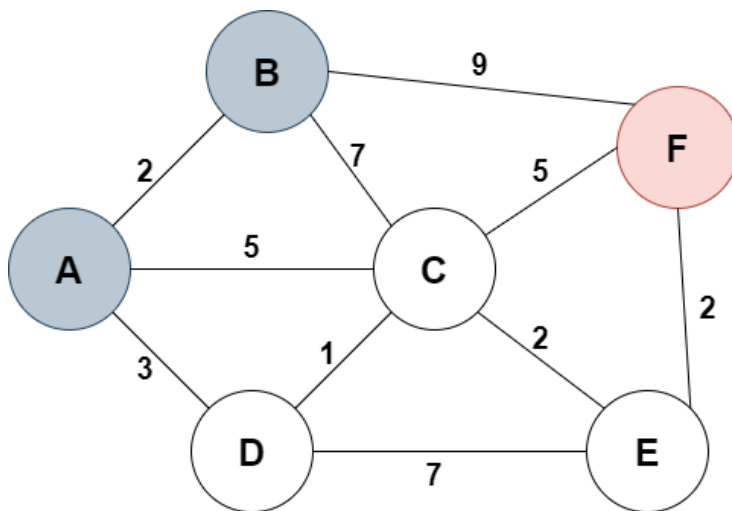
$Q = \{B, C, D, E, F\}$

1.

A노드를 방문해서 이웃 노드의 거리를 계산

A에서 B, C, D의 노드를 방문할 수 있으므로 각각 $d[N]$ 값을 갱신해주고 S에 A노드를 추가

S 집합에 추가된 노드는 이미 방문했다는 의미이므로 다음 탐색에 사용하지 않음



$S = \{A, B\}$

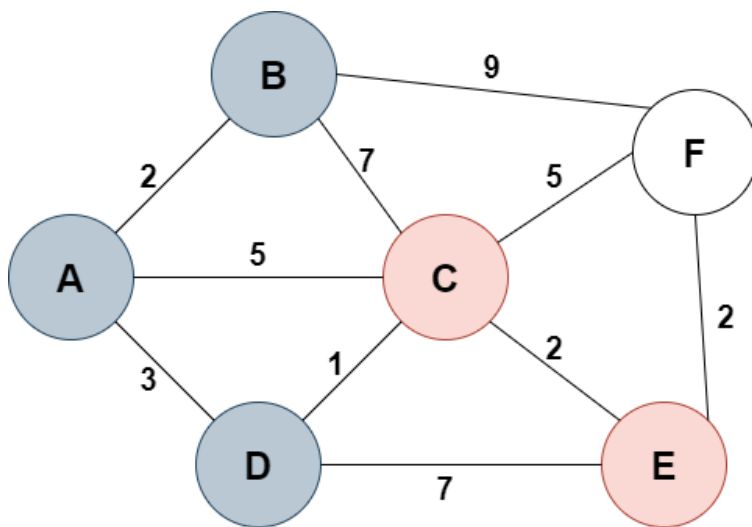
$d[A] = 0$
 $d[B] = 2$
 $d[C] = 5$
 $d[D] = 3$
 $d[E] = \text{INF}$
 $d[F] = 11$

$Q = \{C, D, E, F\}$

2.

다음 탐색은 Q집합 중 $d[N]$ 값이 가장 적은 B를 기준으로 탐색

F를 가는 새 경로가 탐색되었기 때문에 $d[F]$ 값을 갱신해주고, C를 가는 경로는 $2+7 > 5$ 이므로 갱신하지 않음



$S = \{A, B, D\}$

$d[A] = 0$
 $d[B] = 2$
 $d[C] = 4$
 $d[D] = 3$
 $d[E] = 10$
 $d[F] = 11$

$Q = \{C, E, F\}$

3.

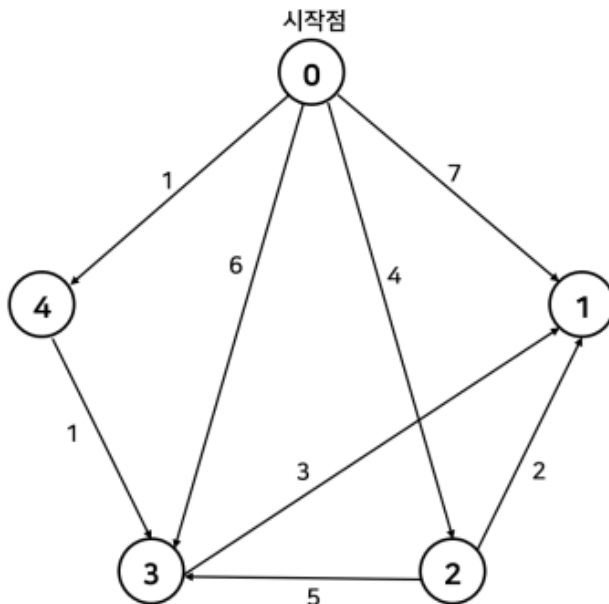
다음 Q집합 중 $d[N]$ 값이 적은 D를 기준으로 탐색

C 경로의 값이 $3 + 1 < 5$ 이므로 $d[C]$ 를 4로 갱신해주고, $d[E]$ 값도 갱신하여 줄

이러한 과정을 반복하면 $d[F]$ 의 최단 경로가 나오게 되며, 모든 노드를 탐색하면 최단 경로는 A -> D -> C -> E -> F = 8이 나오게 됨

2-1-3. 기본 다익스트라 구현[C++]

구현 시 필요한 요소: 비용 배열, 방문 노드 배열, 시작점 ~ 노드 비용 배열



```
weight = { [ 0  7  4  6  1 ],
            [ INF 0  INF INF INF ],
            [ INF 2  0  5  INF ],
            [ INF 3  INF 0  INF ],
            [ INF INF INF 1  0 ] }
```

```
      [0] [1] [2] [3] [4]
visit = { 0, 0, 0, 0, 0 }

dist = { 7, 4, 6, 1 }
```

Weight: [0][0], [0][1], [0][2]... 순서의 비용

Visit: 0이면 방문 X(위 예시에서 Q 집합), 1이면 방문 O(위 예시에서 S 집합)

Dist: 위 예시에서 d[N]의 값

실제 코드

```
int weight[N][N] = {
    {0, 7, 4, 6, 1},
    {INF, 0, INF, INF, INF},
    {INF, 2, 0, 5, INF},
    {INF, 3, INF, 0, INF},
    {INF, INF, INF, 1, 0},
};

bool visit[N];
int dist[N];
int min_node;
```

Weight, visit, dist 배열 선언(전역)

```

void dijkstra(int start)
{
    for (int i = 0; i < N; i++)
    {
        dist[i] = weight[start][i];
    }
    visit[start] = true;

    for (int repeat = 0; repeat < N - 1; repeat++)
    {
        min_node = get_small_node(); // 최소비용노드 탐색
        visit[min_node] = true;      // 최소비용노드 방문처리
        for (int i = 0; i < N; i++)
        {
            // 방문하지 않은 노드 중에, 시작점이 최소비용노드를 경유하는게
            // 더 가까우면 값 갱신
            if (!visit[i])
            {
                if (dist[min_node] + weight[min_node][i] < dist[i])
                    dist[i] = dist[min_node] + weight[min_node][i];
            }
        }
    }
}

```

시작 지점의 경로를 탐색한 뒤 N-1번만큼 반복(시작 지점 제외)

```

int get_small_node()
{
    // 최소비용노드 탐색 함수
    int min = INF;
    int minpos = 0;
    for (int i = 0; i < N; i++)
    {
        if (dist[i] < min && !visit[i])
        {
            min = dist[i];
            minpos = i;
        }
    }
    return minpos;
}

```

최소 비용 노드의 index를 반환

위와 같이 구현 시 시간 복잡도 $O(V^2)$ (V는 노드 개수)

2-1-4. 우선순위 큐(priority queue) [다익스트라 구현\[C++\]](#)

(*우선순위 큐: 우선순위가 높은 데이터가 먼저 나가는 형태의 자료구조)

우선순위 큐를 사용하여 시간 복잡도를 $O(E \log V)$ (V 는 노드 개수, E 는 간선 개수)로 설정하는 방법
노드 개수와 간선 개수에 따라서 우선순위 큐로 구현한 경우가 유리한 경우가 있고, 위의 기본
구현이 유리한 경우가 있음.

구현 시 필요한 요소: 노드 정보 벡터, dist(위에 설명 되어있음), 우선순위 큐 pq

1. 노드 정보 벡터 nodes

각 노드마다 연결되어 있는 모든 노드와 비용 정보를 나타내는 벡터다.

`pair<int, int>`를 원소로 가지며 원소 형태는 {비용, 목적지 노드} 이다.

`nodes[1] = {{5,2}, {8,3}}` // 노드1 -> 노드2 (비용 = 5) , 노드1 -> 노드3 (비용 = 8)

3. 우선순위 큐 pq

간선 정보를 저장한다.

`pair<int, int>`를 원소로 가지며 원소 형태는 {비용, 목적지 노드} 이다.

원소.first 기준 내림차순으로 pop 되는 특징이 있다.

다른 버전의 `get_small_node()` 역할을 대신한다.

알고리즘

1. Dist를 INF로 초기화하고 노드 정보를 입력

2. `Dist[시작 노드] = 0`으로 설정

3. Pq에 {0, 시작 노드} push

4. Pq가 빌 때까지 다음을 반복

`Pq.top() = {비용, 노드}`에 대해서

a. `Dist[노드] < 비용` 이면 skip (이미 앞서 더 짧은 거리를 탐색했기 때문에)

b. 노드의 모든 인접 노드 검사

자신을 거쳐가는 게 빠르면 비용 갱신 후 {갱신한 비용, 인접 노드}를 pq에 push

위와 같이 구현 시 시간 복잡도는 $O(E \log V)$

(우선순위 큐(힙 구현)의 최 시간 복잡도는 $O(\log N)$)

따라서 노드 추출 연산 시간 복잡도는 $O(V * \log V)$ (모든 노드가 한번씩 pq에서 꺼내어지는 경우),

간선 완화 연산 시간 복잡도는 $O(E * \log V)$ (모든 간선이 확인되어지는 경우)

따라서 둘이 합하면 $O((V+E) \log V)$ 임

노드와 간선 개수의 관계는 $E \geq V-1$ ($E+1 \geq V$)임(모든 노드가 이어져 있는 경우)

$V+E$ 에 ($E+1 \geq V$)을 대입하면 $V+E \leq 2E+1$ 이므로 $O(V+E)\log V = O((2E+1)\log V)$ 임

Big-O 표기법에 따라 최고차항만 남기고 앞의 상수 배율은 무시함

따라서 상수가 제거되고 E의 최고차항인 2가 사라지므로 결과적으로 $O(E\log V)$ 가 됨

실제 코드

```
#define INF 1000000000
#define N 3
using namespace std;

vector<pair<int, int>> nodes[N];
int dist[N];
```

비용과 목적 노드를 담은 벡터 형식 선언

```
int main()
{
    for (int i = 0; i < N; i++)
        dist[i] = INF;

    nodes[0].push_back({5, 1});
    nodes[0].push_back({1, 2});

    nodes[2].push_back({3, 1});

    dijkstra(2);

    for (int i = 0; i < N; i++)
        cout << dist[i] << " ";

    return 0;
}
```

Dist를 inf로 초기화하고 노드 정보를 입력

```
void dijkstra(int start)
{
    priority_queue<pair<int, int>> pq;

    pq.push({0, start});
    dist[start] = 0;
}
```

Dist[시작 노드]를 0으로 설정하고, {0, 시작 노드}를 pq에 push

```
while (!pq.empty())
{
    int cost = -pq.top().first; // 우선순위큐는 내림차순 정렬이기 때문
    int here = pq.top().second;
    pq.pop();

    // 이미 최단거리 정보 있으면 pass
    if (dist[here] < cost)
        continue;

    for (int i = 0; i < nodes[here].size(); i++)
    {
        int via_cost = cost + nodes[here][i].first;

        // here를 경유해서 가는게 빠르면 dist 갱신 후 push
        if (via_cost < dist[nodes[here][i].second])
        {
            dist[nodes[here][i].second] = via_cost;
            pq.push({-via_cost, nodes[here][i].second});
            // 내림차순 정렬이라서 최소값이 가장 앞에 오도록 하기 위해 음수로 변환
        }
    }
}
```

While pq가 빌 때까지 조건에 따라 검사를 반복

2-1-5. 다익스트라 한계

- 한번 탐색한 노드는 다시 탐색하지 않는 방법을 사용하고 있기 때문에 간선 가중치가 음수인 경우 해당 알고리즘을 사용할 수 없음
- 모든 경우의 수를 추정함

2-2. A*(A Star) 알고리즘

- 다익스트라 알고리즘을 확장하여 만들어진 경로 탐색 알고리즘
- 목적지까지 가는 가장 빠른(or 비용이 가장 적은) 경로를 찾는 알고리즘
- 목적지까지의 경로를 추정하기 위해 가중치를 설정하는데 이를 휴리스틱이라고 함

추정값 $f(x) = h(x) + g(x)$

$f(x)$: 출발 노드에서 목적지 노드까지 경로의 총 가중치 합

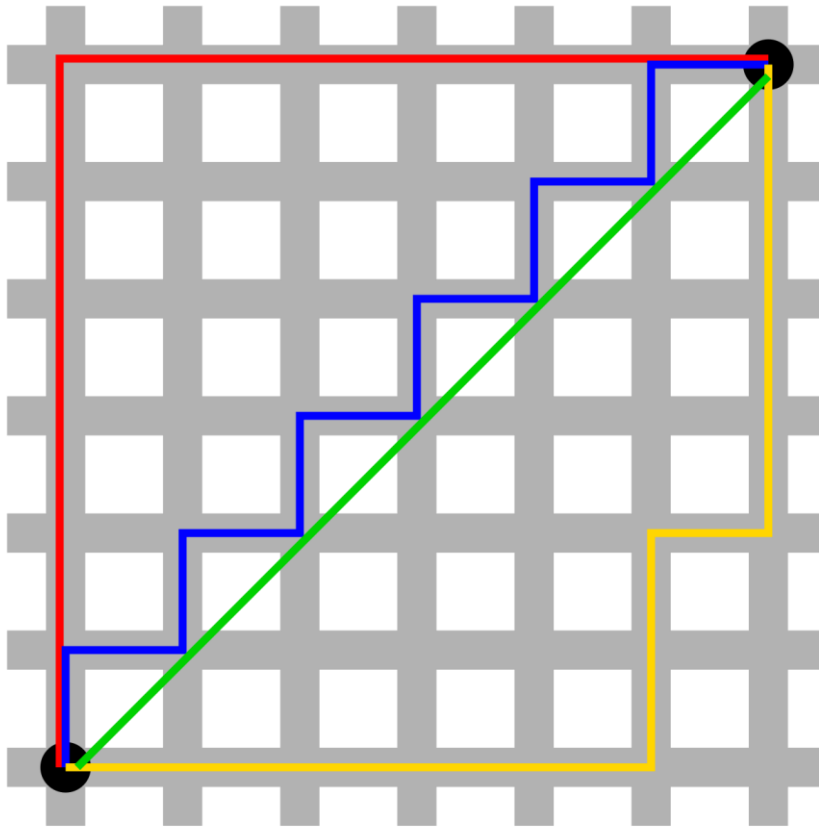
$h(x)$: 현재 노드에서 목적지 노드까지의 추정 경로 가중치(*휴리스틱 함수)

$g(x)$: 현재 노드에서 출발 지점까지의 경로 가중치(지나온 길의 합)

$f(x)$ 가 최소가 되는 노드를 다음 탐색 노드로 선정함(우선순위 큐 사용)

*휴리스틱 함수 $h(x)$

- 실제 목표까지의 거리를 완벽히 계산하지 않고, 그럴 듯하게 예측하여 비용을 계산하는 함수
- 해당 휴리스틱 함수 설정에 따라 성능이 좌우됨
- 대표적인 휴리스틱 값 계산 방법은 맨해튼 거리, 유클리드 거리 두 가지가 있음
 - i) 맨해튼 거리: x, y축으로만 이동할 수 있다고 가정한 거리(파란색 선(=빨간색, 노란색))
 - ii) 유클리드 거리: 피타고라스 정리를 이용해 두 점 사이의 변위를 구한 거리(초록색 선)



휴리스틱 조건

- 허용성(Admissble): $h(n) \leq h^*(n)$ ($h^*(n)$ 은 n 에서 목표까지의 실제 최단 거리)

휴리스틱은 실제 비용보다 과대평가하지 않아야 함(실제보다 낙관적이어야 함)

- 일관성(Consistent): $h(n) \leq \text{cost}(n, n') + h(n')$

$\text{cost}(n, n')$: 노드 n 에서 n' 까지의 실제 비용

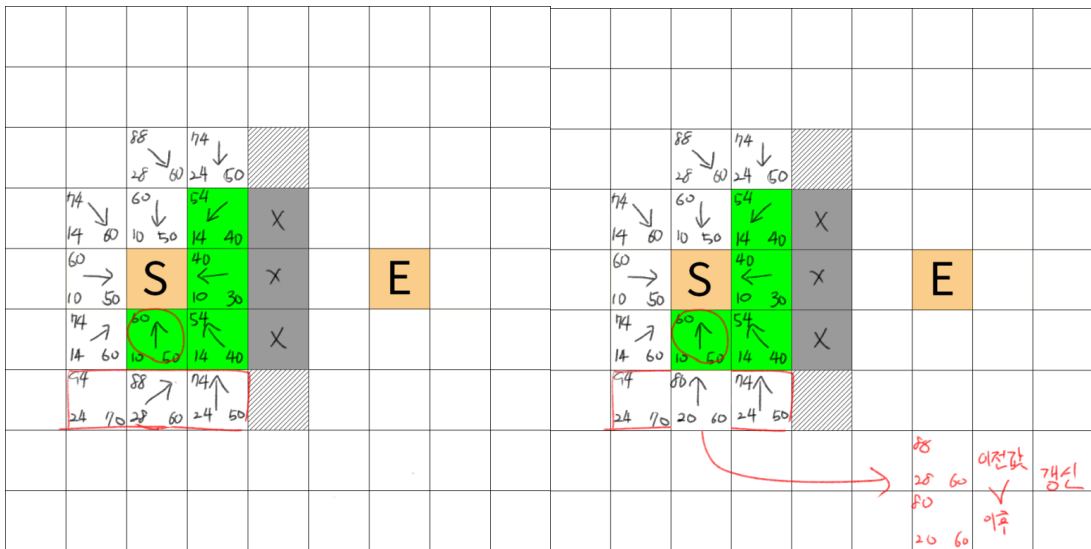
$h(n')$: 노드 n' 에서 목표까지 추정 비용

위의 허용성을 만족하는 조건, $h(n')$ 은 동일한 휴리스틱 함수이고, 휴리스틱 함수는 실제 비용보다 낙관적이어야 하므로 이를 성립해야 함

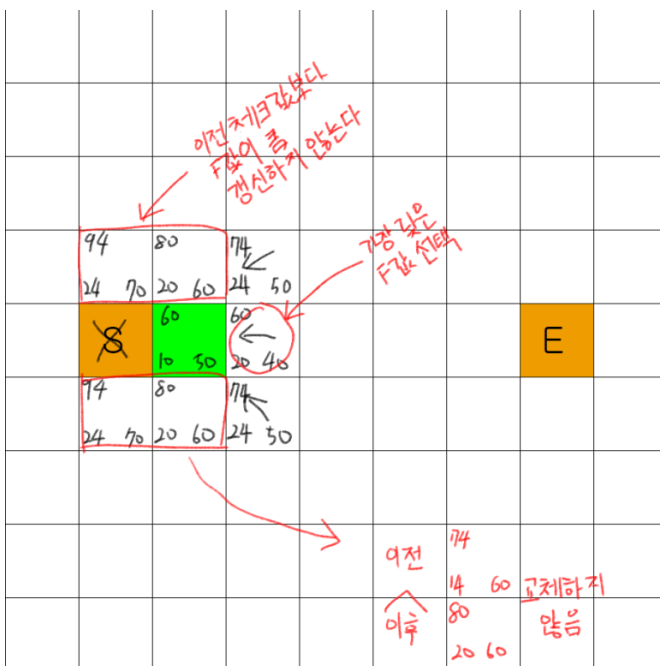
A* 알고리즘 순

1. 탐색을 해야 하는 경로 list(OpenList)와 탐색이 끝난 경로를 담는 list(CloseList)를 설정

2. 시작 지점에서 주위를 탐색해 OpenList에 삽입
3. 탐색한 위치에서 $f(x)$ 를 계산하고 방향을 기록함
4. 시작점을 CloseList에 넣고 OpenList에 있는 경로 중 $f(x)$ 가 가장 작은 경로를 탐색
5. 탐색 위치에서의 주위를 다시 OpenList에 넣음, 막힌 길이나 CloseList는 제외
6. 탐색한 위치에서 $f(x)$ 를 계산하고, 이때 이전에 계산한 값과 중복된 경로가 있다면 $f(x)$ 를 비교하여 작은 값을 적용



7. 현재 위치를 CloseList에 넣고 다시 OpenList에 있는 경로 중 $f(x)$ 가 가장 작은 경로를 탐색



8. 5 – 7 반복, 목적지에 도달했을 경우 방향을 역추적하여 시작점까지 가면 최단 경로가 됨
(목적지에 도달하지 못했을 경우 탐색 실패)